Homework 3: Algorithms for Logistic Regression

Baijia Xu

2026-02-24

GitHub repository: `https://github.com/BXU69/bios731_hw3_xu`

For a given subject in a study, we are interested in modeling $\pi_i = P(Y_i = 1|X_i = x_i)$, where $Y_i \in \{0, 1\}$. The logistic regression model takes the form

$$\text{logit}(\pi_i) = \log\left(\frac{\pi_i}{1-\pi_i}\right) = x_i^T\beta$$

where $x_i = (1, x_{i1}, \ldots, x_{ip})^T$ and $\beta = (\beta_0, \beta_1, \ldots, \beta_p)^T$.

Problem 1: Newton's method

Let $\eta_i = x_i^T\beta$ and $\pi_i = \dfrac{\exp(\eta_i)}{1+\exp(\eta_i)}$.

Likelihood:

$$L(\beta) = \prod_{i=1}^{n} \pi_i^{y_i}(1-\pi_i)^{1-y_i}$$

Log-likelihood:

$$\ell(\beta) = \sum_{i=1}^{n}[y_i log(\pi_i) + (1-y_i)log(1-\pi_i)] = \sum_{i=1}^{n}[y_i\eta_i - \log\{1+\exp(\eta_i)\}]$$

Gradient:

$$\nabla\ell(\beta) = \sum_{i=1}^{n}(y_i - \pi_i)x_i = X^T(y-\pi)$$

Hessian:

$$\nabla^2\ell(\beta) = -\sum_{i=1}^{n}\pi_i(1-\pi_i)x_ix_i^T = -X^TWX,$$

where $W = \text{diag}(\pi_i(1-\pi_i))$.

Newton update for maximizing $\ell(\beta)$:

$$\beta^{(k+1)} = \beta^{(k)} - \left[\nabla^2 \ell\left(\beta^{(k)}\right)\right]^{-1} \nabla \ell\left(\beta^{(k)}\right)$$

Equivalent working form:

$$\beta^{(k+1)} = \beta^{(k)} + \left(X^T W^{(k)} X\right)^{-1} X^T (y - \pi^{(k)}).$$

Is logistic regression convex? If the objective is to minimize negative log-likelihood, yes: $-\ell(\beta)$ is convex since $\nabla^2(-\ell) = X^T W X \succeq 0$. Equivalently, $\ell(\beta)$ is concave.

Problem 2: MM

Let $\eta_i = X_i^T \theta$ and $\eta_i^{(k)} = X_i^T \theta^{(k)}$.

(A).

Define $\phi(u) = -\log(1+u)$ for $u > 0$. $\phi'(u) = -\frac{1}{1+u}$. Since

$$\phi''(u) = \frac{1}{(1+u)^2} > 0,$$

ϕ is convex. Therefore, by the tangent inequality for convex functions,

$$\phi(u) \geq \phi(u_0) + \phi'(u_0)(u - u_0).$$

Set $u = \exp(\eta_i)$ and $u_0 = \exp(\eta_i^{(k)})$. Then

$$-\log\{1 + \exp(\eta_i)\} \geq -\log\{1 + \exp(\eta_i^{(k)})\} - \frac{\exp(\eta_i) - \exp(\eta_i^{(k)})}{1 + \exp(\eta_i^{(k)})},$$

with equality at $\theta = \theta^{(k)}$.

(B).

Write

$$\exp(\eta_i) = \exp(\eta_i^{(k)}) \exp\left(\sum_{j=1}^{p} X_{ij}(\theta_j - \theta_j^{(k)})\right).$$

Using AM-GM with $a_j = \exp\{pX_{ij}(\theta_j - \theta_j^{(k)})\}$,

$$\exp\left(\sum_{j=1}^{p} X_{ij}(\theta_j - \theta_j^{(k)})\right) = \prod_{j=1}^{p} a_j^{1/p} \leq \frac{1}{p} \sum_{j=1}^{p} a_j.$$

Substituting this into part (A), and including the linear term in the log-likelihood, gives

$$g(\theta \mid \theta^{(k)}) = -\frac{1}{p} \sum_{i=1}^{n} \frac{\exp(X_i^T \theta^{(k)})}{1 + \exp(X_i^T \theta^{(k)})} \sum_{j=1}^{p} \exp\{pX_{ij}(\theta_j - \theta_j^{(k)})\} + \sum_{i=1}^{n} Y_i X_i^T \theta + \text{const}.$$

(C)

Differentiate $g(\theta \mid \theta^{(k)})$ with respect to θ_j and set to zero:

$$\nabla g = -\frac{1}{p}\sum_{i=1}^{n} \frac{\exp(X_i^T\theta^{(k)})}{1+\exp(X_i^T\theta^{(k)})} \exp(pX_{ij}(\theta_j - \theta_j^{(k)}))(pX_{ij}) + \sum_{i=1}^{n} Y_i X_{ij}$$

$$= -\sum_{i=1}^{n} \frac{\exp(X_i^T\theta^{(k)})X_{ij}\exp(-pX_{ij}\theta_j^{(k)})}{1+\exp(X_i^T\theta^{(k)})} \exp(pX_{ij}\theta_j) + \sum_{i=1}^{n} Y_i X_{ij} = 0,$$

for each $j = 1, \ldots, p$.

Problem 3: simulation

```r
set.seed(731)

n <- 200
beta_true <- c(1, 0.3)
x <- rnorm(n)
X <- cbind(1, x)
eta <- as.vector(X %*% beta_true)
prob <- 1 / (1 + exp(-eta))
y <- rbinom(n, 1, prob)

loglik <- function(beta, X, y) {
  eta <- as.vector(X %*% beta)
  sum(y * eta - log1p(exp(eta)))
}

score <- function(beta, X, y) {
  eta <- as.vector(X %*% beta)
  mu <- 1 / (1 + exp(-eta))
  as.vector(t(X) %*% (y - mu))
}

hess <- function(beta, X, y) {
  eta <- as.vector(X %*% beta)
  mu <- 1 / (1 + exp(-eta))
  w <- mu * (1 - mu)
  -crossprod(X * w, X)
}

newton_logit <- function(X, y, beta0 = c(0, 0), tol = 1e-8, maxit = 200) {
  beta <- beta0
  t0 <- proc.time()[3]
  for (it in 1:maxit) {
    g <- score(beta, X, y)
    H <- hess(beta, X, y)
    beta_new <- beta - solve(H, g)
    if (max(abs(beta_new - beta)) < tol) {
      beta <- beta_new
```

```r
      break
    }
    beta <- beta_new
  }
  elapsed <- proc.time()[3] - t0

  eta <- as.vector(X %*% beta)
  mu <- 1 / (1 + exp(-eta))
  w <- mu * (1 - mu)
  vc <- solve(crossprod(X * w, X))
  se <- sqrt(diag(vc))

  list(beta = beta, se = se, vcov = vc, iter = it, time = elapsed)
}

mm_logit <- function(X, y, beta0 = c(0, 0), tol = 1e-8, maxit = 500, inner_maxit = 200) {
  p_dim <- ncol(X)
  beta <- beta0
  t0 <- proc.time()[3]

  for (it in 1:maxit) {
    beta_k <- beta
    eta_k <- as.vector(X %*% beta_k)
    w <- exp(eta_k) / (1 + exp(eta_k))
    beta_new <- beta_k

    for (j in 1:p_dim) {
      b <- beta_k[j]
      xj <- X[, j]
      s1 <- sum(y * xj)

      for (inner in 1:inner_maxit) {
        exp_term <- exp(p_dim * xj * (b - beta_k[j]))
        d1 <- -sum(w * xj * exp_term) + s1
        d2 <- -p_dim * sum(w * xj^2 * exp_term)
        b_new <- b - d1 / d2
        if (abs(b_new - b) < 1e-10) {
          b <- b_new
          break
        }
        b <- b_new
      }
      beta_new[j] <- b
    }

    if (max(abs(beta_new - beta_k)) < tol) {
      beta <- beta_new
      break
    }
    beta <- beta_new
  }

  elapsed <- proc.time()[3] - t0
```

```r
  eta <- as.vector(X %*% beta)
  mu <- 1 / (1 + exp(-eta))
  w <- mu * (1 - mu)
  vc <- solve(crossprod(X * w, X))
  se <- sqrt(diag(vc))

  list(beta = beta, se = se, vcov = vc, iter = it, time = elapsed)
}

ci95 <- function(beta, se) {
  cbind(low = beta - 1.96 * se, high = beta + 1.96 * se)
}

fit_newton <- newton_logit(X, y, beta0 = c(0, 0), tol = 1e-8)
fit_mm <- mm_logit(X, y, beta0 = c(0, 0), tol = 1e-8)

t_glm <- system.time(
  fit_glm <- glm(y ~ x, family = binomial())
)
beta_glm <- coef(fit_glm)
se_glm <- sqrt(diag(vcov(fit_glm)))

negll <- function(beta, X, y) -loglik(beta, X, y)
t_optim <- system.time(
  fit_optim <- optim(c(0, 0), negll, X = X, y = y,
                     method = "BFGS", hessian = TRUE,
                     control = list(reltol = 1e-8, maxit = 1000))
)
beta_optim <- fit_optim$par
se_optim <- sqrt(diag(solve(fit_optim$hessian)))

results <- data.frame(
  method = c("Newton", "MM", "glm", "optim_BFGS"),
  beta0 = c(fit_newton$beta[1], fit_mm$beta[1], beta_glm[1], beta_optim[1]),
  beta1 = c(fit_newton$beta[2], fit_mm$beta[2], beta_glm[2], beta_optim[2]),
  iter = c(fit_newton$iter, fit_mm$iter, fit_glm$iter, fit_optim$counts["function"]),
  time_sec = c(fit_newton$time, fit_mm$time, t_glm["elapsed"], t_optim["elapsed"])
)

ci_df <- rbind(
  data.frame(method = "Newton", param = c("beta0", "beta1"), ci95(fit_newton$beta, fit_newton$se)),
  data.frame(method = "MM", param = c("beta0", "beta1"), ci95(fit_mm$beta, fit_mm$se)),
  data.frame(method = "glm", param = c("beta0", "beta1"), ci95(beta_glm, se_glm)),
  data.frame(method = "optim_BFGS", param = c("beta0", "beta1"), ci95(beta_optim, se_optim))
)

results_disp <- results
results_disp[sapply(results_disp, is.numeric)] <-
  round(results_disp[sapply(results_disp, is.numeric)], 6)

ci_df_disp <- ci_df
ci_df_disp[sapply(ci_df_disp, is.numeric)] <-
  round(ci_df_disp[sapply(ci_df_disp, is.numeric)], 6)
```

```r
rownames(ci_df_disp) <- NULL

knitr::kable(results_disp, caption = "Method comparison: estimates, iterations, and time")
```

Table 1: Method comparison: estimates, iterations, and time

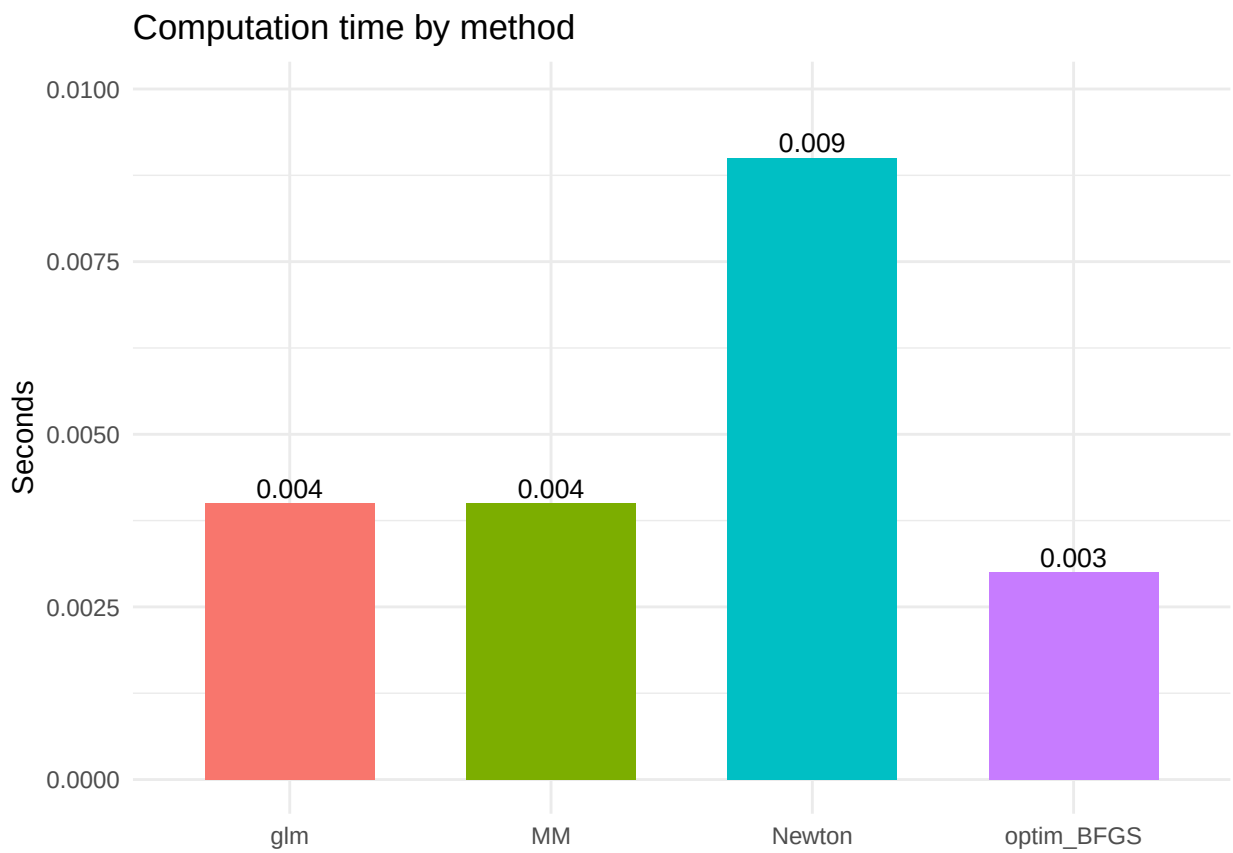
method	beta0	beta1	iter	time_sec
Newton	1.076739	0.174718	5	0.009
MM	1.076739	0.174718	124	0.004
glm	1.076739	0.174718	4	0.004
optim_BFGS	1.076739	0.174718	24	0.003

```r
knitr::kable(ci_df_disp, row.names = FALSE, caption = "95% confidence intervals for beta0 and beta1")
```

Table 2: 95% confidence intervals for beta0 and beta1

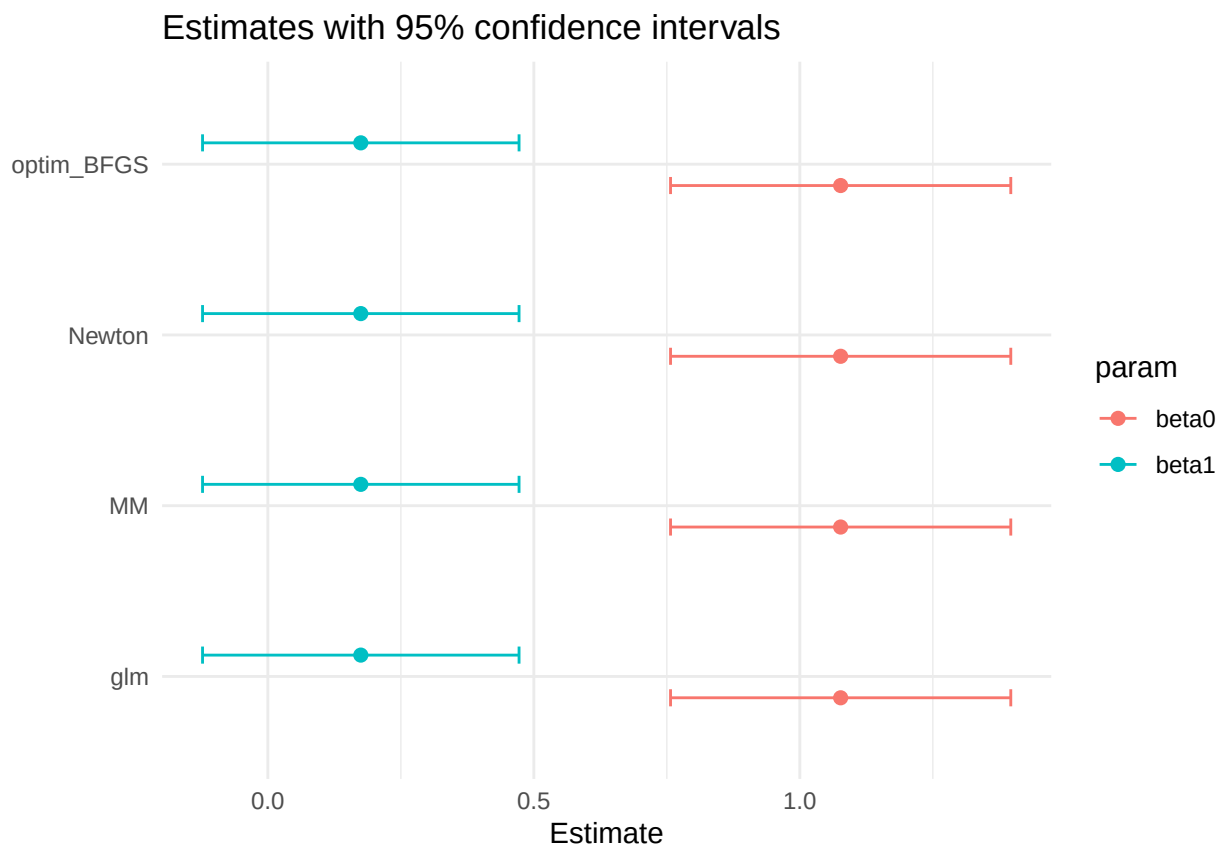
method	param	low	high
Newton	beta0	0.756938	1.396539
Newton	beta1	-0.122769	0.472205
MM	beta0	0.756938	1.396539
MM	beta1	-0.122769	0.472205
glm	beta0	0.756938	1.396539
glm	beta1	-0.122769	0.472205
optim_BFGS	beta0	0.756938	1.396539
optim_BFGS	beta1	-0.122769	0.472205

```r
ggplot(results, aes(x = method, y = time_sec, fill = method)) +
  geom_col(width = 0.65, show.legend = FALSE) +
  geom_text(aes(label = sprintf("%.3f", time_sec)), vjust = -0.3, size = 3.5) +
  labs(title = "Computation time by method", x = NULL, y = "Seconds") +
  theme_minimal(base_size = 11) +
  expand_limits(y = max(results$time_sec) * 1.10)
```




```r
plot_df <- merge(
  data.frame(method = results$method, beta0 = results$beta0, beta1 = results$beta1),
  ci_df,
  by = "method"
)
plot_df$estimate <- ifelse(plot_df$param == "beta0", plot_df$beta0, plot_df$beta1)

ggplot(plot_df, aes(x = estimate, y = method, color = param)) +
  geom_point(position = position_dodge(width = 0.5), size = 2) +
  geom_errorbarh(aes(xmin = low, xmax = high),
                 position = position_dodge(width = 0.5), height = 0.2) +
  labs(title = "Estimates with 95% confidence intervals", x = "Estimate", y = NULL) +
  theme_minimal(base_size = 11)
```



All four methods produce essentially identical point estimates and confidence intervals on this simulated dataset. Newton and `glm()` converge in very few iterations, MM requires more outer iterations, and `optim(BFGS)` is similarly fast in this low-dimensional setting. The numerical agreement confirms all implementations are targeting the same MLE.

Problem 4: EM algorithm for censored exponential data

Assume $T_i \sim \text{Exponential}(\lambda)$ and observed data are (y_i, δ_i) where $y_i = \min(T_i, c_i)$ and $\delta_i = I(T_i \leq c_i)$.

Complete-data log-likelihood:

$$\ell_c(\lambda) = n \log \lambda - \lambda \sum_{i=1}^{n} T_i.$$

E-step:

$$Q(\lambda \mid \lambda^{(k)}) = n \log \lambda - \lambda \sum_{i=1}^{n} E[T_i \mid y_i, \delta_i, \lambda^{(k)}].$$

To compute the conditional expectation in the E-step, separate cases by censoring status.

- If $\delta_i = 1$ (not censored), then T_i is observed exactly, so $T_i = y_i$ and

$$E[T_i \mid y_i, \delta_i = 1, \lambda^{(k)}] = y_i.$$

- If $\delta_i = 0$ (right-censored), then we only know $T_i > y_i$. For an exponential variable,

$$P(T_i > s + t \mid T_i > s) = P(T_i > t),$$

so the residual life after time s has the same distribution as a fresh $\mathrm{Exp}(\lambda)$ variable. Equivalently,

$$T_i \mid (T_i > y_i, \lambda^{(k)}) \overset{d}{=} y_i + Z_i, \quad Z_i \sim \mathrm{Exp}(\lambda^{(k)}).$$

Therefore,

$$E[T_i \mid y_i, \delta_i = 0, \lambda^{(k)}] = y_i + E[Z_i] = y_i + \frac{1}{\lambda^{(k)}}.$$

Combining both cases,

$$E[T_i \mid y_i, \delta_i, \lambda^{(k)}] = \begin{cases} y_i, & \delta_i = 1, \\ y_i + \frac{1}{\lambda^{(k)}}, & \delta_i = 0. \end{cases}$$

M-step:

$$\lambda^{(k+1)} = \frac{n}{\sum_{i=1}^{n} E[T_i \mid y_i, \delta_i, \lambda^{(k)}]}.$$

```r
library(survival)

veteran <- survival::veteran
y_obs <- veteran$time
delta <- veteran$status

em_exp_censored <- function(y, delta, lambda0 = 0.01, tol = 1e-10, maxit = 10000) {
  lambda <- lambda0
  trace <- data.frame(iter = 0, lambda = lambda0)
  t0 <- proc.time()[3]

  for (it in 1:maxit) {
    Et <- ifelse(delta == 1, y, y + 1 / lambda)
    lambda_new <- length(y) / sum(Et)
    trace <- rbind(trace, data.frame(iter = it, lambda = lambda_new))

    if (abs(lambda_new - lambda) < tol * max(1, abs(lambda))) {
      lambda <- lambda_new
      break
    }
    lambda <- lambda_new
  }

  list(lambda = lambda, iter = it, time = proc.time()[3] - t0, trace = trace)
}

fit_em <- em_exp_censored(y_obs, delta)

# Wald CI from observed information: I(lambda) = d/lambda^2
```

```r
d <- sum(delta)
se_lambda <- fit_em$lambda / sqrt(d)
ci_wald <- fit_em$lambda + c(-1, 1) * 1.96 * se_lambda

# AFT comparison using exponential errors
fit_aft <- survreg(Surv(time, status) ~ 1, data = veteran, dist = "exponential")
mu_hat <- as.numeric(coef(fit_aft)[1])
se_mu <- sqrt(fit_aft$var[1, 1])
lambda_aft <- exp(-mu_hat)
ci_lambda_aft <- exp(-(mu_hat + c(1, -1) * 1.96 * se_mu))

summary_tbl <- data.frame(
  quantity = c("EM lambda", "AFT-transformed lambda", "EM iterations", "EM time (sec)"),
  value = c(fit_em$lambda, lambda_aft, fit_em$iter, fit_em$time)
)

ci_tbl <- data.frame(
  method = c("EM/MLE Wald CI", "AFT transformed CI"),
  lower = c(ci_wald[1], ci_lambda_aft[1]),
  upper = c(ci_wald[2], ci_lambda_aft[2])
)

summary_tbl_disp <- summary_tbl
summary_tbl_disp[sapply(summary_tbl_disp, is.numeric)] <-
  round(summary_tbl_disp[sapply(summary_tbl_disp, is.numeric)], 6)

ci_tbl_disp <- ci_tbl
ci_tbl_disp[sapply(ci_tbl_disp, is.numeric)] <-
  round(ci_tbl_disp[sapply(ci_tbl_disp, is.numeric)], 6)

knitr::kable(summary_tbl_disp, caption = "Estimated lambda and convergence summary")
```

Table 3: Estimated lambda and convergence summary

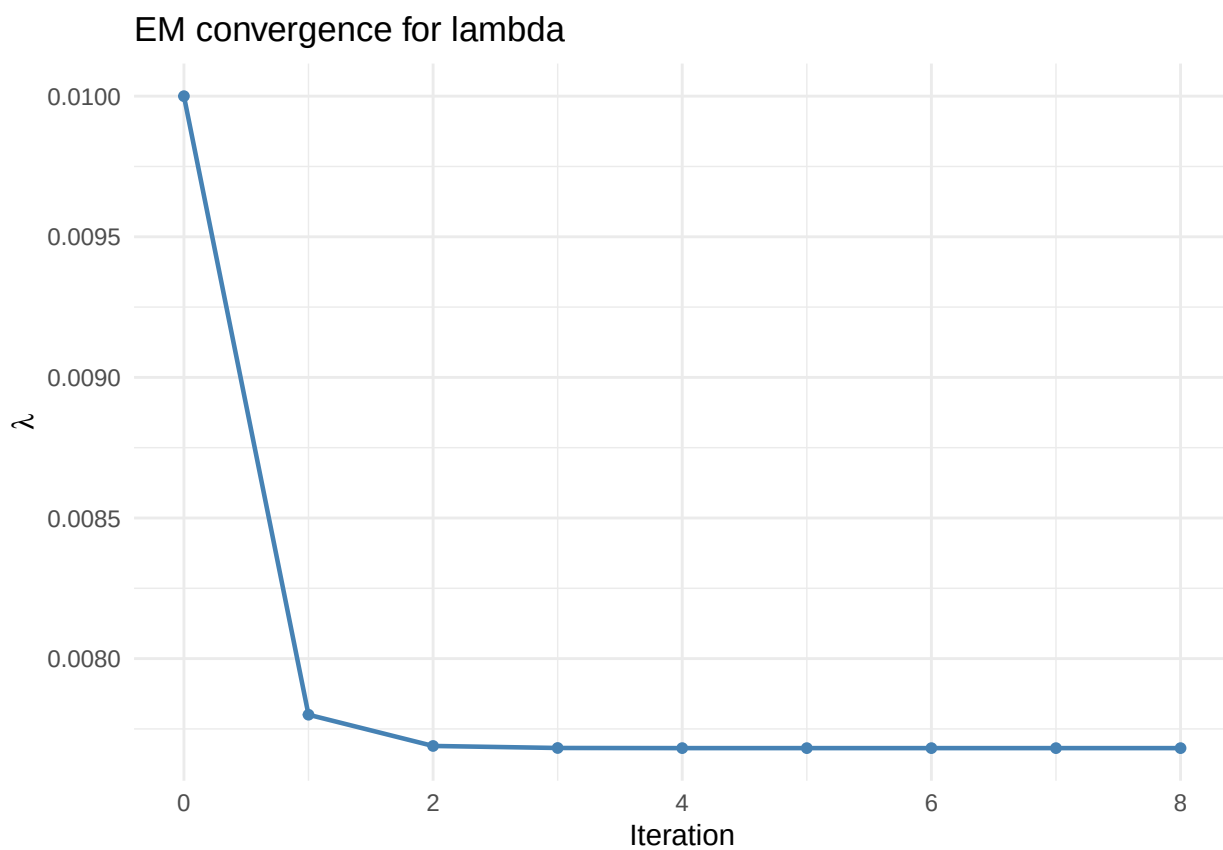
quantity	value
EM lambda	0.007682
AFT-transformed lambda	0.007682
EM iterations	8.000000
EM time (sec)	0.001000

```r
knitr::kable(ci_tbl_disp, caption = "95% confidence intervals for lambda")
```

Table 4: 95% confidence intervals for lambda

method	lower	upper
EM/MLE Wald CI	0.006351	0.009012
AFT transformed CI	0.006460	0.009135

```r
ggplot(fit_em$trace, aes(iter, lambda)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  geom_point(color = "steelblue", size = 1.4) +
  labs(title = "EM convergence for lambda", x = "Iteration", y = expression(lambda)) +
  theme_minimal(base_size = 11)
```



Convergence was monitored by parameter change: $|\lambda^{(k+1)} - \lambda^{(k)}| < \text{tol} \cdot \max(1, |\lambda^{(k)}|)$ with `tol = 1e-10`.

The 95% CI for λ is obtained using a Wald approximation based on observed information for the censored exponential model: with $d = \sum_i \delta_i$, $I(\lambda) = d/\lambda^2$, so $\text{SE}(\hat{\lambda}) \approx \hat{\lambda}/\sqrt{d}$ and

$$\hat{\lambda} \pm 1.96\,\text{SE}(\hat{\lambda}).$$

For the AFT model, we use the normal-based CI from `survreg` on the log-time scale and transform back to λ via $\lambda = \exp(-\mu)$.

Extra credit (up to 5 points)

Part A: logistic regression

For logistic regression, expected and observed information are equivalent:

$$-\nabla^2 \ell(\beta) = X^T W X.$$

This Hessian depends on β (and X) through π_i, but not directly on y_i, so its expectation equals itself.

Part B: probit regression

For probit regression, expected and observed information are generally not equal in finite samples. Probit is not the canonical Bernoulli link, and the observed Hessian includes data-dependent curvature terms. The expected information averages those terms, so it differs from observed information except asymptotically or in special cases.